BABSON

SIEM Homelab + Custom Local AI Security Analyst

Complete Step-by-Step Master Guide v7 — Custom Python Wazuh MCP Server + Goose Desktop + Ollama + Qwen3 8B

October 06, 2026

SIEM Homelab + Custom Local AI Security Analyst

Complete Step-by-Step Master Guide v7

Custom Python Wazuh MCP Server + Goose Desktop + Ollama + Qwen3 8B

Updated: October 6, 2026
Platform: Apple M2 Mac with 16 GB unified memory
Virtualization: VMware Fusion Pro 13.6.4 or compatible newer release
SIEM: Wazuh all-in-one deployment
Local AI: Ollama + Qwen3 8B
MCP client: Goose Desktop
Custom integration: Locally maintained Python Wazuh MCP server using STDIO transport

---

Project Overview

This project builds an isolated security operations homelab in which controlled activity from a Kali Linux attacker is detected by Wazuh and investigated through a local AI analyst. Goose Desktop uses the locally hosted `qwen3:8b` model through Ollama and launches the custom Wazuh MCP server as a local STDIO extension.

The MCP server exposes five read-only tools:

- `list_agents`
- `recent_alerts`
- `search_alerts`
- `vulnerabilities`
- `agent_health`

The server retrieves manager and agent state from the Wazuh Manager API and searches alert and vulnerability documents through the Wazuh Indexer. Indexer access remains private through an SSH tunnel from the Mac to the Wazuh server.

> **WARNING**
>
> This lab intentionally contains vulnerable applications, weak credentials, and offensive-security tools. Keep Kali and the vulnerable target isolated on `vmnet2`. Never bridge either system to a home, school, business, or public network while vulnerable services are enabled.

Project Goals

By the end of the project, you will have:

- A Wazuh all-in-one SIEM server.
- A monitored Ubuntu target with an active Wazuh agent.
- DVWA, OWASP Juice Shop, Apache, ModSecurity, and deliberately weak lab-only SSH credentials.
- An isolated Kali Linux attacker VM.
- A locally hosted Qwen3 8B model through Ollama.
- Goose Desktop configured as the local AI and MCP client.
- A custom Python Wazuh MCP server that you understand and control.
- Five read-only tools that retrieve live Wazuh data.
- A repeatable detection, investigation, and incident-report workflow.
- A foundation for additional aggregation, endpoint-security, reporting, and controlled-response tools.

Design Principles

1. **Local first:** SIEM data, model inference, and analyst prompts remain on the Mac and lab VMs.
2. **Read-only first:** The initial tools retrieve information without modifying Wazuh or endpoints.
3. **No MCP listening port:** Goose starts the MCP server as a local STDIO subprocess.
4. **Private Indexer access:** Port `9200` is reached through an SSH tunnel and is not exposed through the Wazuh VM firewall.
5. **Clear data-source separation:** Manager state comes from the Wazuh Manager API; alerts and vulnerability findings come from the Wazuh Indexer.
6. **Independent validation:** Test Wazuh, the tunnel, Python dependencies, the MCP server, Goose, and the model separately.
7. **Controlled growth:** Add tools only after the initial five operate reliably.
8. **Evidence before inference:** The analyst must distinguish tool-returned facts from interpretation.

Confirmed Lab Architecture

Logical Architecture

```
Goose Desktop
├── Local model provider
│   └── Ollama http://localhost:11434
│       └── qwen3:8b
│
└── Custom STDIO extension: Wazuh Local SOC
    └── Project virtual-environment Python
        └── server.py
            ├── Wazuh Manager API
            │   └── https://192.168.7.10:55000
            │       ├── list_agents
            │       └── agent_health
            │
            └── Local SSH tunnel
                └── https://127.0.0.1:9200
                    └── Wazuh Indexer
                        ├── recent_alerts
                        ├── search_alerts
                        ├── vulnerabilities
                        └── agent_health alert context
```

Goose launches `server.py` and exchanges MCP messages through the subprocess's standard input and output. The MCP server does not bind to port `8000` or expose an HTTP endpoint.

Network Reference

Component	Interface/ network	Address	Purpose
Mac host	Private to my Mac	`192.168.7.1`	Management workstation
Wazuh server	Private to my Mac	`192.168.7.10`	Dashboard, Manager API, and agent traffic
Wazuh server	Share with my Mac	DHCP	Temporary internet access for installation and updates
Target VM	Private to my Mac	`192.168.7.20`	Wazuh agent and Mac management access
Target VM	`vmnet2`	`192.168.200.10`	Vulnerable attack surface
Kali VM	`vmnet2`	`192.168.200.5`	Isolated attacker

Service and Process Reference

Service or process	Host	Port/transport	Purpose
Wazuh Dashboard	Wazuh VM	`443/TCP`	SIEM web interface
Wazuh Manager API	Wazuh VM	`55000/TCP`	Agent and manager information
Wazuh agent events	Wazuh VM	`1514/TCP`	Agent communication
Wazuh enrollment	Wazuh VM	`1515/TCP`	Agent registration
Wazuh Indexer	Wazuh VM	`9200/TCP`, tunnel only	Alert and vulnerability search
Ollama	Mac	`11434/TCP`	Local model inference
Goose Desktop	Mac	Local application	Analyst interface and MCP client
Wazuh Local SOC extension	Mac	STDIO subprocess	Custom MCP tools
DVWA	Target VM	`8080/TCP`	Vulnerable web application
Juice Shop	Target VM	`3000/TCP`	Vulnerable web application

Expected Communication Paths

Source	Destination	Expected result
Mac	Wazuh server management address	Allowed
Mac	Target management address	Allowed
Mac	Ollama on localhost	Allowed
Goose	Custom MCP subprocess	Allowed through STDIO
MCP server	Wazuh Manager API	Allowed
MCP server	Local Indexer tunnel	Allowed while tunnel is active
Kali	Target attack address	Allowed
Kali	Wazuh management address	Blocked or unreachable
Kali	Mac	Blocked or unreachable
Target	Internet	Normally unavailable in final lab state

Phase 0 — Prerequisites and Preparation

Estimated time: 30 minutes

Required Software

- VMware Fusion Pro 13.6.4 or a compatible newer release.
- Ubuntu Server 24.04 LTS ARM64 ISO.
- Kali Linux ARM64 installer image.
- Ollama for macOS.
- Goose Desktop.
- Python 3.11 or later on macOS.
- Git.
- Homebrew, recommended for Goose Desktop installation.
- Existing custom Wazuh MCP project containing the working `server.py` implementation.

Storage Target

Item	Approximate space
Wazuh VM	30 GB
Target VM	30 GB
Kali VM	40 GB
Ollama model	5–7 GB
Snapshots and logs	30–60 GB
Recommended free space before build	180 GB or more

Project Directory Convention

The recommended project location is:

```
~/wazuh-local-mcp
```

This guide refers to that location as `<PROJECT_DIR>`. If the working project is stored elsewhere, retain it and substitute its actual absolute path.

Determine the absolute project path with:

```
cd /path/to/your/project
pwd
```

> **TIP**
>
> A path without spaces is the most reliable option for a local STDIO extension. If the project is inside a directory such as `Homelab SIEM`, configure Goose with the Python executable and script as separate values and use complete absolute paths.

Phase 0 Completion Check

- VMware Fusion is installed.
- All guest images are ARM64/AArch64.
- Ubuntu 24.04 LTS is selected for the Wazuh and Target VMs.

- Sufficient disk space is available.
- Goose Desktop, Ollama, Python, and Git can be installed on the Mac.
- The working custom MCP project is available and backed up.

Phase 1 — VMware Networking

Estimated time: 20 minutes

Network Definitions

VMware label	Function	Lab use
Share with my Mac	NAT	Wazuh installation and temporary updates
Private to my Mac	Host-only	Mac, Wazuh, and Target management
`vmnet2`	Custom isolated network	Kali-to-Target attack segment
Bridged	Physical LAN access	Not used

Create `vmnet2`

1. Open **VMware Fusion → Settings → Network**.
2. Unlock the settings.
3. Select **+** to create a custom network.
4. Confirm that `vmnet2` appears.
5. Do not enable internet sharing for `vmnet2`.
6. Apply the configuration.

Phase 1 Completion Check

- **Private to my Mac** exists for management.
- `vmnet2` exists as an isolated custom network.
- Kali has no NAT or bridged adapter.
- The vulnerable Target interface on `vmnet2` has no default route to the internet.

Phase 2 — Wazuh Server VM

Estimated time: 60 minutes

VM Configuration

Setting	Value
OS	Ubuntu Server 24.04 LTS ARM64
RAM	4096 MB
CPU	2 cores
Disk	30 GB
Adapter 1	Private to my Mac
Adapter 2	Share with my Mac
Hostname	`wazuh-server`
Username	`wazuh`

Static Management Address

Create `/etc/netplan/50-cloud-init.yaml`:

```
sudo tee /etc/netplan/50-cloud-init.yaml <<'EOF'
network:
  version: 2
  ethernets:
    ens160:
      dhcp4: false
      addresses:
        - 192.168.7.10/24
    ens192:
      dhcp4: true
EOF

sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo bash -c 'echo "network: {config: disabled}" > /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg'
sudo netplan apply
```

Verify:

```
ip address show ens160
ip address show ens192
```

If interface names differ, identify them with `ip link` and adjust the Netplan file before applying it.

Firewall

```
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw allow 55000/tcp
sudo ufw allow 1514/tcp
sudo ufw allow 1515/tcp
sudo ufw enable
sudo ufw status verbose
```

> **WARNING**
>
> Do not open port `9200` through UFW. The Mac reaches the Indexer only through the SSH tunnel.

Wazuh Installation

Follow the official Wazuh quickstart for the supported all-in-one installation method. Preserve:

- Generated dashboard credentials.

- Wazuh API credentials.
- Wazuh Indexer credentials.
- The installation archive or password file.
- Any generated certificate material.

Verify:

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

Manager API Test

Obtain the `wazuh-wui` password from the Wazuh Dashboard configuration:

```
sudo cat /usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml
```

From the Mac, authenticate and test the API:

```
TOKEN=$(curl -k -s \
  -u 'wazuh-wui:REPLACE_WITH_PASSWORD' \
  -X POST \
  'https://192.168.7.10:55000/security/user/authenticate?raw=true')

curl -k -s \
  -H "Authorization: Bearer $TOKEN" \
  'https://192.168.7.10:55000/' | python3 -m json.tool
```

Do not save a live token in the guide, screenshots, or Git repository.

Phase 2 Completion Check

- The Wazuh Dashboard loads at `https://192.168.7.10`.
- Manager, Indexer, and Dashboard services are active.
- The Manager API accepts JWT authentication.
- Port `9200` is not exposed through UFW.
- Wazuh credentials and installation artifacts are stored securely.

Phase 3 — Target VM and Wazuh Agent

Estimated time: 30 minutes

VM Configuration

Setting	Value
OS	Ubuntu Server 24.04 LTS ARM64
RAM	4096 MB
CPU	2 cores
Disk	30 GB
Adapter 1	Private to my Mac
Adapter 2	`vmnet2`
Hostname	`target-vm`
Username	`labuser`

Network Configuration

```
sudo tee /etc/netplan/50-cloud-init.yaml <<'EOF'
network:
  version: 2
  ethernets:
    ens160:
      dhcp4: false
      addresses:
        - 192.168.7.20/24
    ens192:
      dhcp4: false
      addresses:
        - 192.168.200.10/24
EOF

sudo chmod 600 /etc/netplan/50-cloud-init.yaml
sudo bash -c 'echo "network: {config: disabled}" > /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg'
sudo netplan apply
```

Verify:

```
ip address show ens160
ip address show ens192
ping -c 3 192.168.7.10
ip route
```

The management network should provide access to Wazuh. The isolated attack interface should expose only the target services to Kali.

Install the Wazuh Agent

Install the current compatible Wazuh agent from the official Wazuh repository. Configure the manager address as:

```
192.168.7.10
```

Enroll and start the agent. Verify:

```
sudo systemctl status wazuh-agent
sudo tail -50 /var/ossec/logs/ossec.log
```

Confirm in the Wazuh Dashboard that `target-vm` is Active.

Phase 3 Completion Check

- The Target has `192.168.7.20` and `192.168.200.10`.
- The Target reaches the Wazuh server through the management network.
- The Wazuh agent service is active.
- The agent appears as Active in Wazuh.

Phase 4 — Vulnerable Target Stack

Estimated time: 45 minutes

Install and configure the following components on the isolated Target:

- Apache2
- ModSecurity
- OWASP Core Rule Set
- DVWA
- OWASP Juice Shop
- Docker
- Password-based SSH for the controlled exercise
- A deliberately weak lab-only account

Required Logging

Ensure the Wazuh agent monitors the relevant logs. Add appropriate `<localfile>` entries to the agent configuration:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/apache2/modsec_audit.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/error.log</location>
</localfile>
```

Validate the agent configuration and restart it:

```
sudo systemctl restart wazuh-agent
sudo systemctl status wazuh-agent
sudo tail -50 /var/ossec/logs/ossec.log
```

Service Verification

From Kali or another device on the isolated segment when appropriate:

```
DVWA:       http://192.168.200.10:8080
Juice Shop: http://192.168.200.10:3000
```

Confirm that Apache and ModSecurity generate logs before continuing.

Baseline Snapshot

Create a VMware snapshot named:

```
Clean Baseline - Phase 4 Complete
```

Phase 4 Completion Check

- Apache and ModSecurity are active.
- DVWA loads at `http://192.168.200.10:8080`.
- Juice Shop loads at `http://192.168.200.10:3000`.

- Relevant log files exist and receive events.
- Wazuh monitors the required logs.
- A clean Target snapshot exists.

Phase 5 — Kali Linux Attacker VM

Estimated time: 20 minutes

VM Configuration

Setting	Value
OS	Kali Linux ARM64
RAM	4096 MB
CPU	2 cores
Disk	40 GB
Network adapter	`vmnet2` only
Address	`192.168.200.5/24`

Connectivity Test

```
ping -c 3 192.168.200.10
ping -c 3 192.168.7.10
```

Expected result:

- Target ping succeeds.
- Wazuh management address is unreachable.
- Kali has no route to the Mac management interface or internet.

Inspect the routing table:

```
ip address
ip route
```

Tool Check

```
nmap --version
hydra -h | head
nikto -Version
sqlmap --version
```

Baseline Snapshot

Create a snapshot named:

```
Kali Clean - Phase 5 Complete
```

Phase 5 Completion Check

- Kali has one adapter on `vmnet2`.
- Kali can reach `192.168.200.10`.
- Kali cannot reach the Wazuh management network or Mac.
- Required security-testing tools are available.

Phase 6 — Validate the Existing Custom Python Wazuh MCP Server

Estimated time: 45–60 minutes

This guide assumes the custom server implementation already exists and is working. The authoritative implementation is:

```
<PROJECT_DIR>/server.py
```

Refer to `server.py` in the project directory. Do not recreate or overwrite it from this guide.

6.1 Understand the Two Wazuh Data Sources

Tool	Backend	Purpose
`list_agents`	Wazuh Manager API	Agent inventory and connection state
`agent_health`	Manager API + Indexer	Agent state plus recent alert activity
`recent_alerts`	Wazuh Indexer	Recent indexed alert documents
`search_alerts`	Wazuh Indexer	Alert content search
`vulnerabilities`	Wazuh Indexer	Vulnerability state documents

The Manager API is the correct source for manager and agent inventory. The Indexer is the correct searchable source for alerts and vulnerability findings.

6.2 Verify the Project

```
cd <PROJECT_DIR>
pwd
ls -la
```

Expected core files:

```
<PROJECT_DIR>/
├── .env
├── .gitignore
├── .venv/
├── requirements.txt
└── server.py
```

The project may contain tests, logs, or supporting modules in addition to these files.

6.3 Activate the Virtual Environment

```
cd <PROJECT_DIR>
source .venv/bin/activate
python --version
python -m pip list
```

If the environment must be rebuilt:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Do not unnecessarily upgrade working dependencies during an investigation session. Test upgrades separately and retain a known-good requirements file.

6.4 Protect Secrets

Confirm that `.gitignore` excludes at least:

```
.venv/
.env
__pycache__/
*.pyc
.DS_Store
.pytest_cache/
```

Set restrictive permissions:

```
chmod 600 .env
ls -l .env
```

Expected owner permissions:

```
-rw-------
```

The existing working server uses the environment-variable names implemented in `server.py`. Preserve those names. In particular, do not replace the working `WAZUH_PASS` variable with `WAZUH_PASSWORD`, and do not replace `VERIFY_SSL` with `WAZUH_VERIFY_SSL`.

Typical configuration categories include:

Category	Purpose
Manager API URL	Wazuh Manager API endpoint
Manager API username	API service account
`WAZUH_PASS`	Password expected by the working server
Indexer URL	Local tunnel endpoint, normally `https://127.0.0.1:9200`
Indexer username/password	Indexer search credentials
Alert index pattern	Normally a `wazuh-alerts-*` pattern
Vulnerability index pattern	Normally a `wazuh-states-vulnerabilities-*` pattern
`VERIFY_SSL`	TLS verification behavior

> **WARNING**
>
> Never paste the contents of `.env` into Goose, an LLM prompt, a screenshot, a ticket, or a public repository.

6.5 Start the Indexer SSH Tunnel

In a dedicated Mac terminal:

```
ssh -N \
  -o ServerAliveInterval=30 \
  -o ExitOnForwardFailure=yes \
  -L 9200:127.0.0.1:9200 \
  wazuh@192.168.7.10
```

Leave this terminal running while using Indexer-backed tools.

Check the local port:

```
lsof -nP -iTCP:9200 -sTCP:LISTEN
```

Test the tunnel:

```
curl -k https://127.0.0.1:9200
```

An authentication response proves that the tunnel reached the Indexer. `Connection refused` usually means the tunnel is absent, the remote Indexer is unavailable, or local port `9200` is already in use.

6.6 Discover Index Names

With the tunnel running:

```
curl -k \
  -u 'INDEXER_USER:REPLACE_WITH_INDEXER_PASSWORD' \
  'https://127.0.0.1:9200/_cat/indices?v'
```

Look for alert and vulnerability indices. Common patterns are:

```
wazuh-alerts-*
wazuh-states-vulnerabilities-*
```

Use the actual names present in the deployment and ensure the working `.env` matches them.

6.7 Validate Python Syntax

```
cd <PROJECT_DIR>
source .venv/bin/activate
python -m py_compile server.py
```

No output means syntax validation passed.

6.8 Validate Imports

```
python -c "import server; print('server import successful')"
```

If import-time configuration requires the tunnel or credentials, start the tunnel and verify `.env` first.

6.9 Preserve STDIO Transport

The working server is configured for STDIO transport so Goose can launch it directly. Do not change it back to a Streamable HTTP listener.

STDIO operational rules:

- Do not add ordinary debug `print()` statements to stdout.

- Send diagnostics to stderr or a protected log file.
- Do not print startup banners to stdout.
- Do not add interactive `input()` prompts.
- Do not print credentials, JWTs, authorization headers, or complete secrets.
- Keep protocol output separate from human-readable diagnostics.

6.10 Optional Independent MCP Validation

Before involving the model, validate that the server starts, advertises the five tools, and can execute at least `list_agents` through an MCP-capable test client or the existing known-good validation method.

Expected tool inventory:

```
list_agents
recent_alerts
search_alerts
vulnerabilities
agent_health
```

The project has reached the required baseline when tool discovery succeeds and `list_agents` returns live agent data.

Phase 6 Completion Check

- The project directory contains the working `server.py`.
- The virtual-environment interpreter imports the server successfully.
- `.env` is excluded from Git and has mode `600`.
- The working environment-variable names are preserved.
- The Indexer SSH tunnel listens locally on port `9200`.
- Manager and Indexer credentials are valid.
- Alert searching succeeds.
- Vulnerability searching succeeds or returns a valid empty result.
- The server uses STDIO and exposes no MCP network port.
- Five tools are discoverable.
- `list_agents` returns live Wazuh agent data.

Phase 7 — Ollama and Goose Desktop Integration

Estimated time: 45 minutes

7.1 Install and Verify Ollama

Install the Ollama macOS application or use an approved local installation method. Start Ollama and verify the local API:

```
curl http://localhost:11434/api/tags
```

A JSON response confirms that Ollama is listening.

7.2 Pull Qwen3 8B

```
ollama pull qwen3:8b
ollama list
```

Test the model:

```
ollama run qwen3:8b \
  "You are a SOC analyst. Explain Wazuh in one sentence."
```

Exit the interactive model session when complete.

7.3 Install Goose Desktop

Install Goose Desktop with Homebrew:

```
brew install --cask block-goose
```

Launch Goose Desktop from `/Applications` or Spotlight.

7.4 Configure the Ollama Provider

In Goose Desktop:

1. Open **Settings**.
2. Open the provider configuration.

3. Select or configure **Ollama**.
4. Set the API host to:

```
http://localhost:11434
```

1. Select or enter the model:

```
qwen3:8b
```

1. Save the provider configuration.
2. Start a simple chat and confirm that the model responds before enabling the Wazuh extension.

7.5 Determine Absolute Extension Paths

From Terminal:

```
cd <PROJECT_DIR>
PYTHON_PATH="$(pwd)/.venv/bin/python"
SERVER_PATH="$(pwd)/server.py"

printf 'Python: %s\n' "$PYTHON_PATH"
printf 'Server: %s\n' "$SERVER_PATH"

test -x "$PYTHON_PATH" && echo "Python executable found"
test -f "$SERVER_PATH" && echo "server.py found"
```

Copy the two printed absolute paths. Do not use `~` in the Goose extension configuration if an absolute path is available.

7.6 Configure the Custom STDIO Extension

In Goose Desktop:

1. Open the sidebar.
2. Select **Extensions**.
3. Add a custom extension.
4. Choose **STDIO** as the extension type.
5. Use the following values:

Field	Value
Name	`Wazuh Local SOC`
Description	Read-only Wazuh agent, alert, vulnerability, and health tools
Type	`STDIO`
Command	Absolute path to `<PROJECT_DIR>/.venv/bin/python`
Arguments	Absolute path to `<PROJECT_DIR>/server.py`
Enabled	Yes

Example with the recommended project location:

```
Command:
/Users/YOUR_USERNAME/wazuh-local-mcp/.venv/bin/python

Argument:
/Users/YOUR_USERNAME/wazuh-local-mcp/server.py
```

> **IMPORTANT**
>
> Configure the Python executable as the command and `server.py` as a separate argument. Do not enter `source .venv/bin/activate && python server.py` as the command.

If the project path contains spaces, keep each path in its own Goose field. If Goose still reports `os error 2`, move or clone the project to a no-space path such as `~/wazuh-local-mcp`, recreate the virtual environment if needed, and update the extension paths.

7.7 Validate Tool Discovery

Enable the extension and confirm that Goose discovers exactly these tools:

- `list_agents`
- `recent_alerts`
- `search_alerts`
- `vulnerabilities`
- `agent_health`

If discovery fails, inspect the Goose extension error and validate the command manually:

```
"/absolute/path/to/.venv/bin/python" \
  "/absolute/path/to/server.py"
```

A STDIO server may appear to wait silently for a client. Use `Ctrl+C` to stop this manual launch. A traceback indicates a path, dependency, environment, credential, or code problem.

7.8 Recommended SOC Analyst Instructions

Use the following as persistent session instructions, a Goose recipe prompt, or the opening message for an investigation:

> You are a defensive SOC analyst working in an isolated Wazuh homelab. Use the Wazuh Local SOC tools whenever the user asks about agents, alerts, vulnerabilities, health, authentication events, web attacks, or incident activity. Never invent SIEM data. Clearly distinguish facts returned by tools from your analysis. Use timestamps, agent IDs, rule IDs, rule levels, source IPs, destination details, and MITRE ATT&CK data when available. The tools are read-only. If a query returns no results, say so and suggest a wider time range or a different search phrase. Do not claim an action was blocked, remediated, contained, or successful unless the returned evidence proves it.

7.9 Tool Acceptance Tests

Run one test at a time. Explicit tool names improve reliability with a smaller local model.

Test 1 — Agent Inventory

```
Use the list_agents tool. List all Wazuh agents with their ID, name,
IP address, operating system, status, and last keepalive. Do not infer
values that are missing from the tool output.
```

Test 2 — Recent Alerts

```
Use the recent_alerts tool for the last 15 minutes. Return no more than
20 alerts and summarize the highest-severity findings. Separate returned
facts from your analysis.
```

Test 3 — Alert Search

```
Use search_alerts to search the last 60 minutes for SSH authentication
failures. Show timestamps, agent, rule ID, rule level, username, and
source IP when present.
```

Test 4 — Vulnerabilities

```
Use the vulnerabilities tool to list High vulnerabilities for agent 001.
Group the returned results by affected package. If no findings are
returned, state that clearly.
```

Test 5 — Agent Health

```
Use agent_health for agent 001. Explain its connection state and
summarize its latest alerts. Distinguish the Wazuh status from your
analyst assessment.
```

7.10 Diagnose the Correct Layer

Observation	Most likely layer
Ollama does not respond	Ollama process or local API
Goose cannot generate normal text	Provider/model configuration
Extension fails immediately	Command, argument, virtual environment, or path
Goose sees no Wazuh tools	MCP startup or registration
Tools appear but are not selected	Model tool-selection behavior
`list_agents` works but alert tools fail	SSH tunnel, Indexer credentials, or index patterns
Tool calls run but return empty results	Time window, query, or absence of matching data
MCP framing/protocol error	Unexpected stdout output from the server

Phase 7 Completion Check

- Ollama responds on port `11434`.

- `qwen3:8b` is installed.
- Goose Desktop is installed and starts successfully.
- Goose connects to Ollama at `http://localhost:11434`.
- The selected model is `qwen3:8b`.
- `Wazuh Local SOC` is configured as a custom STDIO extension.
- Goose uses the project virtual-environment Python interpreter.
- Goose launches the existing `server.py`.
- All five tools are discovered.
- Every tool has been called successfully at least once.
- No HTTP MCP endpoint or port `8000` is required.

Phase 8 — First Full Detection and Investigation Exercise

Estimated time: 60–90 minutes

Authorized Scope

All attack commands in this phase are restricted to:

```
Attacker: 192.168.200.5
Target:   192.168.200.10
```

> **DANGER**
>
> Do not run these commands against public systems, production systems, school infrastructure, business networks, or systems you do not own and control.

Boot Order

1. Start the Wazuh Server VM.
2. Wait until Wazuh Manager, Indexer, and Dashboard services are ready.
3. Start the Target VM.
4. Confirm that the Wazuh agent is Active.

5. Start or verify Ollama.
6. Start the Indexer SSH tunnel.
7. Open Goose Desktop.
8. Select the Ollama provider and `qwen3:8b`.
9. Enable and verify the `Wazuh Local SOC` extension.
10. Start Kali only when ready to begin the authorized exercise.

Pre-Attack Baseline

In Goose Desktop, ask:

```
Use list_agents and recent_alerts. Confirm agent status and establish an
alert baseline for the last five minutes. Do not include older data.
Clearly identify any alerts already present before the exercise.
```

Create a Target snapshot:

```
Pre-Attack - YYYY-MM-DD
```

Record:

- Exercise start time.
- Kali source IP.
- Target IP.
- Target agent ID and name.
- Baseline alert count.
- Time zone used in the report.

Exercise 1 — Port Scan

From Kali:

```
nmap -sV 192.168.200.10
nmap -A -T4 192.168.200.10
```

Investigation prompt:

```
Use recent_alerts and search_alerts to investigate the last 10 minutes
for evidence of scanning or service discovery on target-vm. Identify the
source IP, destination services, relevant Wazuh rules, and whether the
evidence is conclusive or only suggestive. Do not invent destination
ports or rule mappings that are absent from the returned data.
```

Analyst questions:

- Did Wazuh receive direct scan telemetry or only service-log side effects?
- Is the Kali source IP visible?
- Which services responded?
- Are repeated connections or unusual request patterns present?
- Does the evidence prove scanning, or merely support that hypothesis?

Exercise 2 — SSH Authentication Attack Simulation

Use only the deliberately weak lab account. Stop after enough events have been generated for detection and analysis.

```
hydra -l victim \
  -P /usr/share/wordlists/rockyou.txt \
  ssh://192.168.200.10 \
  -t 4 \
  -V
```

Investigation prompt:

```
Use search_alerts for the last 15 minutes to find SSH authentication
failures on target-vm. Build a timeline, count the returned events, list
usernames and source IPs when available, and determine whether any
successful login is visible. Do not treat missing success evidence as
proof that no login occurred.
```

Record:

- First and last observed failure.
- Source IP.
- Attempted username.
- Number of returned events.
- Wazuh rule IDs and levels.
- Any successful-login event.

- Any evidence gaps caused by query limits or log parsing.

Exercise 3 — Web Application Testing

From Kali:

```
nikto -h http://192.168.200.10:8080
```

Use DVWA manually to generate controlled SQL-injection and command-injection events. Keep all activity inside the isolated lab.

Investigation prompt:

```
Use search_alerts and recent_alerts for the last 20 minutes. Separate
ModSecurity events from Apache access and error events. Identify attack
patterns, targeted paths, source IP, Wazuh rule levels, and available
OWASP or MITRE mappings. Do not invent mappings absent from the data;
label analyst-added mappings separately.
```

Full Incident Report Prompt

```
Use the Wazuh Local SOC tools to investigate all activity on target-vm
during the last 60 minutes. Produce a formal SOC incident report with:

1. Executive Summary
2. Scope and Data Sources
3. Alert Timeline
4. Port-Scanning Evidence
5. SSH Authentication Evidence
6. Web Attack Evidence
7. Indicators of Compromise
8. MITRE ATT&CK Mapping
9. Impact Assessment
10. Confidence and Data Gaps
11. Containment and Remediation Recommendations

Clearly separate tool-returned facts from analyst interpretation. Cite
actual timestamps, agent IDs, rule IDs, levels, source IPs, usernames,
paths, and mappings when returned. Do not claim containment, blocking,
compromise, or successful exploitation unless the evidence proves it.
```

Post-Exercise Cleanup

1. Save the incident report.
2. Export relevant screenshots or alert evidence.
3. Record the query windows and tool parameters used.
4. Revert the Target to the pre-attack or clean snapshot.
5. Suspend Kali.
6. Restart the Target if needed.
7. Verify that the Wazuh agent reconnects.
8. Run `list_agents` and `agent_health` after recovery.

Phase 8 Completion Check

- Each controlled exercise generated observable telemetry.
- The five tools supported at least one investigation.
- The final report cites actual returned evidence.
- Facts and analyst inference are clearly separated.
- Data gaps are documented.
- The Target was reverted to a clean state.
- Kali was suspended after the exercise.

Phase 9 — Session Management SOP

Monitoring Session

1. Start the Wazuh Server VM.
2. Wait for Manager, Indexer, and Dashboard services.
3. Start the Target VM.
4. Confirm the agent is Active.
5. Confirm Ollama:

```
curl http://localhost:11434/api/tags
```

1. Start the Indexer tunnel:

```
ssh -N \
  -o ServerAliveInterval=30 \
  -o ExitOnForwardFailure=yes \
  -L 9200:127.0.0.1:9200 \
  wazuh@192.168.7.10
```

1. Open Goose Desktop.
2. Confirm the Ollama provider and `qwen3:8b` are selected.
3. Confirm `Wazuh Local SOC` is enabled.
4. Test:

```
Use list_agents and report the status of every agent.
```

Penetration-Test Session

Follow the Monitoring Session procedure, then:

1. Close unnecessary Mac applications.
2. Take a Target snapshot.
3. Start Kali.
4. Verify Kali still has only the isolated adapter.
5. Establish a five-minute alert baseline.
6. Record the exercise start time.
7. Begin only the approved lab exercises.

Ending a Session

1. Save investigation notes and reports.
2. Revert the Target if attacks were performed.
3. Suspend or shut down Kali.
4. Confirm the Target agent reconnects if the Target was restarted.
5. Close Goose Desktop.
6. Confirm the Goose-launched MCP subprocess has ended.
7. Stop the SSH tunnel with `Ctrl+C`.
8. Shut down the Target.
9. Shut down the Wazuh Server.
10. Leave Ollama idle or quit it from the menu bar.

Quick Health Commands

On the Mac:

```
curl http://localhost:11434/api/tags
lsof -nP -iTCP:9200 -sTCP:LISTEN
```

On the Wazuh server:

```
sudo systemctl is-active wazuh-manager
sudo systemctl is-active wazuh-indexer
sudo systemctl is-active wazuh-dashboard
```

On the Target:

```
sudo systemctl is-active wazuh-agent
sudo tail -20 /var/ossec/logs/ossec.log
```

Phase 10 — Security Hardening After the MVP

Complete these improvements after all five tools work reliably.

10.1 Create a Read-Only Indexer Account

Replace the Indexer administrator bootstrap credential with a dedicated account restricted to the required indices:

```
wazuh-alerts-*
wazuh-states-vulnerabilities-*
```

Minimum intended capabilities should support only the required read and search operations, such as:

- Read documents.
- Search documents.
- Retrieve necessary index metadata or settings.
- Monitor only what is required for the queries.

After changing credentials:

1. Update the protected `.env`.

2. Restart the Goose extension.
3. Retest all five tools.
4. Verify that write and administrative operations are denied.

10.2 Enable Certificate Verification

The MVP may use:

```
VERIFY_SSL=false
```

A hardened deployment should:

1. Copy the appropriate Wazuh CA certificate to the Mac.
2. Protect the certificate file from unauthorized modification.
3. Configure the Python clients to use the CA bundle.
4. Set the working TLS verification option appropriately.
5. Confirm Manager API and Indexer requests succeed.
6. Remove unnecessary insecure-request warnings only after verification works.

10.3 Keep MCP Local Through STDIO

The standard design exposes no MCP network listener. Preserve that design unless a documented requirement justifies a remote transport.

If a future release uses HTTP:

- Treat it as a separate security architecture.
- Require authentication and authorization.
- Bind to the narrowest interface possible.
- Add TLS.
- Add rate limits and audit logging.
- Validate every tool input.
- Never expose action tools without confirmation and allowlists.

10.4 Protect Secrets

- Never commit `.env`.
- Keep `.env` at mode `600`.

- Use dedicated service accounts.
- Rotate credentials if exposed in screenshots, prompts, tickets, or repositories.
- Do not include passwords or tokens in tool output.
- Do not send secrets to the model.
- Avoid placing credentials directly in the Goose extension command.
- Back up secrets only in an approved password manager or encrypted store.

10.5 Harden the Goose Extension Launch

- Use absolute paths.
- Use the project virtual-environment interpreter.
- Pass `server.py` as a separate argument.
- Avoid shell wrappers and chained commands.
- Restrict write permissions on `server.py`, `.env`, and dependency files.
- Review dependency changes before installation.
- Keep the project in a user-controlled directory.
- Disable the extension when it is not needed.

10.6 Add Safe Audit Logging

Future server logging should record:

- Timestamp.
- Tool name.
- Sanitized input parameters.
- Backend category.
- Backend status code.
- Result count.
- Execution duration.
- Error category.

Do not log:

- Passwords.
- JWTs.
- Authorization headers.
- Complete sensitive event bodies by default.

- Raw `.env` values.
- MCP protocol messages containing sensitive context.

Send diagnostics to stderr or a protected log file rather than stdout.

10.7 Dependency and Change Control

- Keep a known-good `requirements.txt` or lock file.
- Back up the working project before upgrades.
- Test dependency changes outside active investigation sessions.
- Re-run syntax, import, tool-discovery, and functional tests after changes.
- Maintain a rollback path.

Development Roadmap

Release 0.1 — Initial Five Tools

- `list_agents`
- `recent_alerts`
- `search_alerts`
- `vulnerabilities`
- `agent_health`

Release 0.2 — Aggregation Tools

- Alert count by severity.
- Top Wazuh rules.
- Top source IPs.
- Top targeted agents.
- Alert timeline buckets.

Release 0.3 — Endpoint Security Tools

- File integrity monitoring events.
- Security Configuration Assessment results.

- Rootcheck findings.
- Installed packages.
- Listening ports.

Release 0.4 — Reporting Tools

- Incident evidence package.
- IOC extraction.
- MITRE ATT&CK summary.
- Vulnerability prioritization.
- Markdown report export.

Release 0.5 — Controlled Actions

Only after the read-only implementation is stable:

- Trigger an approved agent scan.
- Run allowlisted active-response actions.
- Block an isolated lab IP.
- Restart a lab agent.

> **WARNING**
>
> Every action tool must require explicit confirmation, validate targets against an allowlist, create an audit record, and refuse non-lab addresses.

Troubleshooting Reference

Symptom	Likely cause	Corrective action
Manager API returns `401`	Incorrect API user or password	Recheck the service account and manually test JWT authentication
Dashboard works but API fails	Dashboard login confused with API credentials	Use the configured API service account and JWT flow
`localhost:9200` connection refused	SSH tunnel is not running	Restart the tunnel with `ExitOnForwardFailure=yes`
SSH tunnel starts and exits	Remote Indexer unavailable or local port occupied	Check `wazuh-indexer`; run `lsof -i :9200` on the Mac
Indexer returns `401`	Incorrect Indexer credential	Verify the saved Indexer account credentials
Alert index not found	Index pattern differs	Run `_cat/indices?v` and update the protected configuration
Vulnerability index not found	Vulnerability Detection has not populated it or the name differs	Verify the module and list indices
`ModuleNotFoundError: mcp`	Wrong Python interpreter	Use `<PROJECT_DIR>/.venv/bin/python` and reinstall requirements if necessary
`.env` not loaded	Wrong directory, filename, or implementation expectation	Confirm `.env` is beside `server.py` and inspect the working loader logic
Goose reports `os error 2`	Executable or script path is invalid	Use absolute paths and confirm each file exists
`os error 2` with project path containing spaces	Goose launch parsing or field configuration issue	Use separate command and argument fields or move to a no-space path
Extension fails immediately	Incorrect virtual-environment path or import failure	Run the exact command manually and inspect the traceback
Goose discovers no tools	Server exits during startup or is not using STDIO	Validate syntax, imports, `.env`, and transport
Only some tools appear		

Symptom	Likely cause	Corrective action
	Registration failed partway through startup	Inspect errors and verify every decorated tool loads
Goose sees tools but Qwen does not call them	Tool-selection limitation	Explicitly say `Use the tool_name tool` and test one tool at a time
Model sends invalid arguments	Smaller-model schema limitation	Simplify the prompt and use tool defaults
MCP protocol or framing error	Unexpected stdout output	Remove stdout debug prints and log to stderr or a file
`list_agents` works but alert tools fail	Tunnel, Indexer credentials, or index configuration	Verify `9200`, credentials, and index names
Recent alerts return none	Time window too narrow	Increase the window to 60 or 1440 minutes
Search alerts return none	Query is too restrictive	Use fewer terms, broader phrasing, or a larger time window
Vulnerabilities return none	No populated state documents or filters too narrow	Test without severity/search filters and verify the index
Agent health says unknown	Incorrect agent ID	Run `list_agents` and use the returned ID
High memory pressure	Multiple VMs, Goose, and Qwen running together	Close other apps and start Kali only when required
Tool output overwhelms context	Result limit is too high	Use 20–50 results and narrower filters
Goose extension works only after manual activation	Extension disabled or session did not load it	Enable `Wazuh Local SOC` and begin a fresh session
Credentials work in Terminal but not Goose	Different environment or working-directory assumptions	Ensure `server.py` loads `.env` by absolute project path and use the venv interpreter

Path Diagnostic Checklist

```
cd <PROJECT_DIR>
pwd
ls -l server.py .env .venv/bin/python
.venv/bin/python -m py_compile server.py
.venv/bin/python -c "import server; print('import ok')"
```

Tunnel Diagnostic Checklist

```
lsof -nP -iTCP:9200 -sTCP:LISTEN
curl -k https://127.0.0.1:9200
ssh -v -N \
  -o ExitOnForwardFailure=yes \
  -L 9200:127.0.0.1:9200 \
  wazuh@192.168.7.10
```

Model Diagnostic Checklist

```
curl http://localhost:11434/api/tags
ollama list
ollama run qwen3:8b "Reply with the word ready."
```

Validation Checklist

Infrastructure

- ☐ Wazuh Server is reachable at `192.168.7.10`.
- ☐ Target agent is Active.
- ☐ Kali is isolated on `vmnet2`.
- ☐ Kali cannot reach the management network.
- ☐ Port `9200` is not exposed through UFW.

- [ ] Target and Kali snapshots exist.

Local AI

- [ ] Ollama responds on port `11434`.
- [ ] `qwen3:8b` is installed.
- [ ] Goose Desktop is operational.
- [ ] Goose connects to `http://localhost:11434`.
- [ ] Goose uses `qwen3:8b`.

Custom MCP

- [ ] The project directory contains the working `server.py`.
- [ ] The virtual environment is functional.
- [ ] `.env` is excluded from Git.
- [ ] `.env` has mode `600`.
- [ ] The working `WAZUH_PASS` and `VERIFY_SSL` names are preserved.
- [ ] Manager API access succeeds.
- [ ] Indexer alert search succeeds.
- [ ] Vulnerability search succeeds or returns a valid empty set.
- [ ] The SSH tunnel is active when Indexer tools are used.
- [ ] `Wazuh Local SOC` is configured as STDIO.
- [ ] The extension uses the venv Python executable.
- [ ] The extension passes `server.py` as an argument.
- [ ]

Goose discovers all five tools.

- ☐ Each tool has been called successfully.
- ☐ `list_agents` returns live data.
- ☐ No MCP network port is exposed.

Security

- ☐ All initial tools are read-only.
- ☐ Indexer access uses an SSH tunnel.
- ☐ No credentials appear in Git history.
- ☐ No credentials are exposed to the model.
- ☐ The MCP server writes no diagnostic text to stdout.
- ☐ Kali cannot reach the management network.
- ☐ Action tools have not been added without confirmation and allowlists.

Prompt Library

Agent Inventory

```
Use list_agents. Create a table containing agent ID, name, IP, operating
system, version, status, and last keepalive. Flag any agent that is not
active. Do not invent missing fields.
```

High-Severity Alert Review

```
Use recent_alerts for the last 30 minutes with minimum rule level 10.
Summarize events in descending severity and identify repeated source IPs,
agents, and rules. Distinguish returned facts from pattern analysis.
```

Authentication Investigation

```
Use search_alerts for the last 60 minutes with the query SSH
authentication failure. Build a timestamped timeline and determine
whether any successful authentication followed the failures. If no
success event is returned, describe that as an evidence gap rather than
proof that no login occurred.
```

Web Attack Investigation

```
Use search_alerts for the last 30 minutes with the query ModSecurity.
Group results by attack category, target path, source IP, and Wazuh rule.
Identify the highest-confidence malicious behavior. Label any
analyst-added OWASP or MITRE mapping separately.
```

Vulnerability Review

```
Use vulnerabilities for agent 001. Retrieve Critical findings first,
then High findings. Group them by package and recommend a remediation
order based on severity, exposure, and package function. Do not claim a
package is internet-exposed without supporting evidence.
```

Health Review

```
Use agent_health for agent 001. Report connection state, last keepalive,
operating system, version, latest alerts, and evidence requiring analyst
attention. Separate Wazuh status from your health assessment.
```

Full SOC Report

```
Use the available Wazuh tools to analyze target-vm for the last 60
minutes. Produce a report with Executive Summary, Scope, Data Sources,
Evidence, Timeline, IOCs, MITRE ATT&CK Mapping, Impact, Confidence, Data
Gaps, and Remediation. Clearly distinguish observed evidence from
inference and cite actual returned fields.
```

No-Result Follow-Up

```
The previous tool call returned no results. Do not invent findings.
Repeat the search with a wider time range and a simpler query, then
explain whether the absence of results reflects the data or the search
limitations.
```

Source and Version Notes

This guide relies on the following primary project documentation for components that change over time:

- Wazuh Quickstart
- Wazuh Indexer API
- Wazuh Indexer indices
- Official MCP Python SDK
- Goose installation documentation
- Goose custom extension documentation
- Goose extensions documentation
- Ollama integration with Goose
- Ollama macOS documentation

The tested local configuration documented here is:

- Goose Desktop installed through the `block-goose` Homebrew cask.
- Ollama at `http://localhost:11434`.
- Model `qwen3:8b`.
- A custom Goose STDIO extension named `Wazuh Local SOC`.
- The extension launches `server.py` through the project's Python virtual environment.

- A local SSH tunnel forwards Mac port `9200` to the Wazuh Indexer.
- Five read-only tools are discovered, and `list_agents` returns live agent data.

Because Wazuh, Goose, Ollama, and MCP evolve, consult the official documentation before changing versions, dependency pins, API behavior, or security settings. Preserve the known-good project before upgrading.

Final Project Outcome

After completing this guide, the homelab provides a local, understandable, and controllable security-analysis workflow:

1. Wazuh collects and indexes endpoint and application telemetry.
2. The Mac privately reaches the Indexer through an SSH tunnel.
3. Ollama runs `qwen3:8b` locally.
4. Goose Desktop provides the analyst interface.
5. Goose launches the custom Wazuh MCP server as a STDIO extension.
6. The server exposes five read-only tools.
7. The analyst uses live evidence to investigate controlled lab activity.
8. Findings are documented with timestamps, rules, agents, indicators, confidence, and data gaps.
9. The vulnerable Target is reverted after exercises.
10. The integration can be expanded incrementally without exposing an MCP network service.

The correct operating sequence is:

1. Verify the Wazuh and Target environment.
2. Start Ollama.
3. Start the Indexer SSH tunnel.
4. Open Goose Desktop.
5. Select `qwen3:8b`.
6. Enable `Wazuh Local SOC`.
7. Verify the five tools.
8. Establish a baseline.
9. Run only authorized isolated exercises.
10. Investigate and report using returned evidence.
11. Revert the Target and suspend Kali.

12. Replace bootstrap privileges with least-privilege credentials and enable certificate verification.